



US012418421B2

(12) **United States Patent**
Thubert et al.

(10) **Patent No.:** **US 12,418,421 B2**
(45) **Date of Patent:** **Sep. 16, 2025**

(54) **CLIENT DEVICE VERIFICATION**

(71) Applicant: **Cisco Technology, Inc.**, San Jose, CA (US)

(72) Inventors: **Pascal Thubert**, Roquefort les Pins (FR); **Eric A. Voit**, Bethesda, MD (US); **Eric Levy-Abegnoli**, Valbonne (FR); **Patrick Wetterwald**, Mouans Sartoux (FR); **Jonas Zaddach**, Antibes (FR)

(73) Assignee: **Cisco Technology, Inc.**, San Jose, CA (US)

(*) Notice: Subject to any disclaimer, the term of this patent is extended or adjusted under 35 U.S.C. 154(b) by 56 days.

(21) Appl. No.: **18/237,583**

(22) Filed: **Aug. 24, 2023**

(65) **Prior Publication Data**

US 2025/0070980 A1 Feb. 27, 2025

(51) **Int. Cl.**

G06F 15/16 (2006.01)
H04L 9/32 (2006.01)
H04L 61/2503 (2022.01)

(52) **U.S. Cl.**

CPC **H04L 9/3236** (2013.01); **H04L 61/2503** (2013.01)

(58) **Field of Classification Search**

CPC H04L 61/2503; H04L 9/3236; H04L 61/4511; H04L 63/0421; H04L 9/3239; H04L 9/0643
USPC 713/168; 709/245–246
See application file for complete search history.

(56) **References Cited**

U.S. PATENT DOCUMENTS

8,201,252 B2 *	6/2012	Chuah	H04L 63/1458
			713/189
9,755,834 B1 *	9/2017	Johnson	H04L 9/3236
9,781,214 B2 *	10/2017	Torok	H04L 61/2503
10,218,698 B2 *	2/2019	Yin	H04L 9/0643
11,316,820 B2 *	4/2022	Stahl	H04L 61/103
11,924,179 B2 *	3/2024	Recio	H04L 63/0428
12,021,998 B2 *	6/2024	Bettger	H04L 9/3239
2004/0054924 A1	3/2004	Chuah et al.	
2011/0145420 A1 *	6/2011	Kakadia	H04L 67/1027
			709/227
2012/0054869 A1	3/2012	Yen et al.	
2014/0250240 A1 *	9/2014	Schell	H04L 61/2535
			709/245
2015/0373044 A1	12/2015	Stiansen et al.	
2016/0127413 A1	5/2016	Kraitsman et al.	

(Continued)

Primary Examiner — Bharat Barot

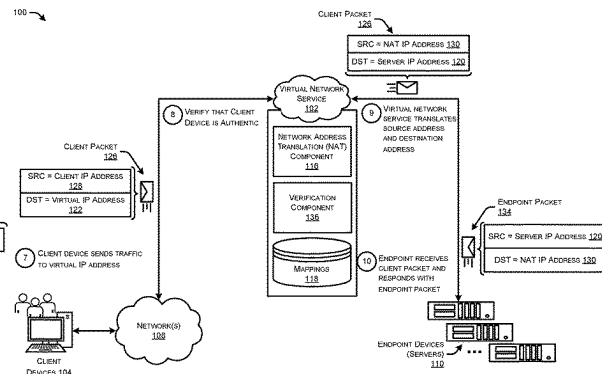
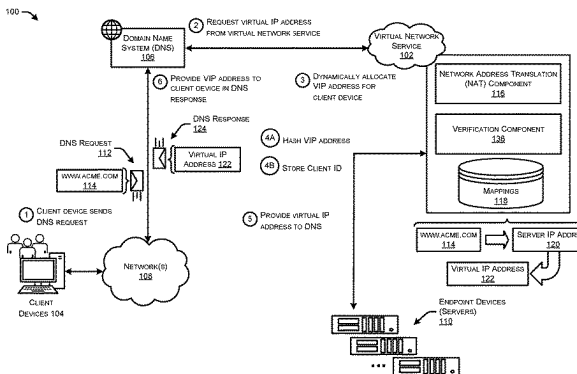
(74) *Attorney, Agent, or Firm* — Lee & Hayes, P.C.

(57)

ABSTRACT

Techniques for using Network Address Translation (NAT), Mobile Internet Protocol (MIP), and/or other techniques in conjunction with Domain Name System (DNS) to anonymize server-side addresses in data communications and verify an authenticity of a client device attempting to use a virtual IP (VIP) address. Rather than having DNS provide a client device with an IP address of an endpoint device, such as a server, the DNS instead returns a VIP address that is mapped to the client device and the endpoint device. The client device may then communicate data packets to the server using the VIP address as the destination address, and a virtual network service that works in conjunction with DNS can verify an authenticity of the client device and convert the VIP address to the actual IP address of the server using NAT and forward the data packet onto the server.

20 Claims, 5 Drawing Sheets



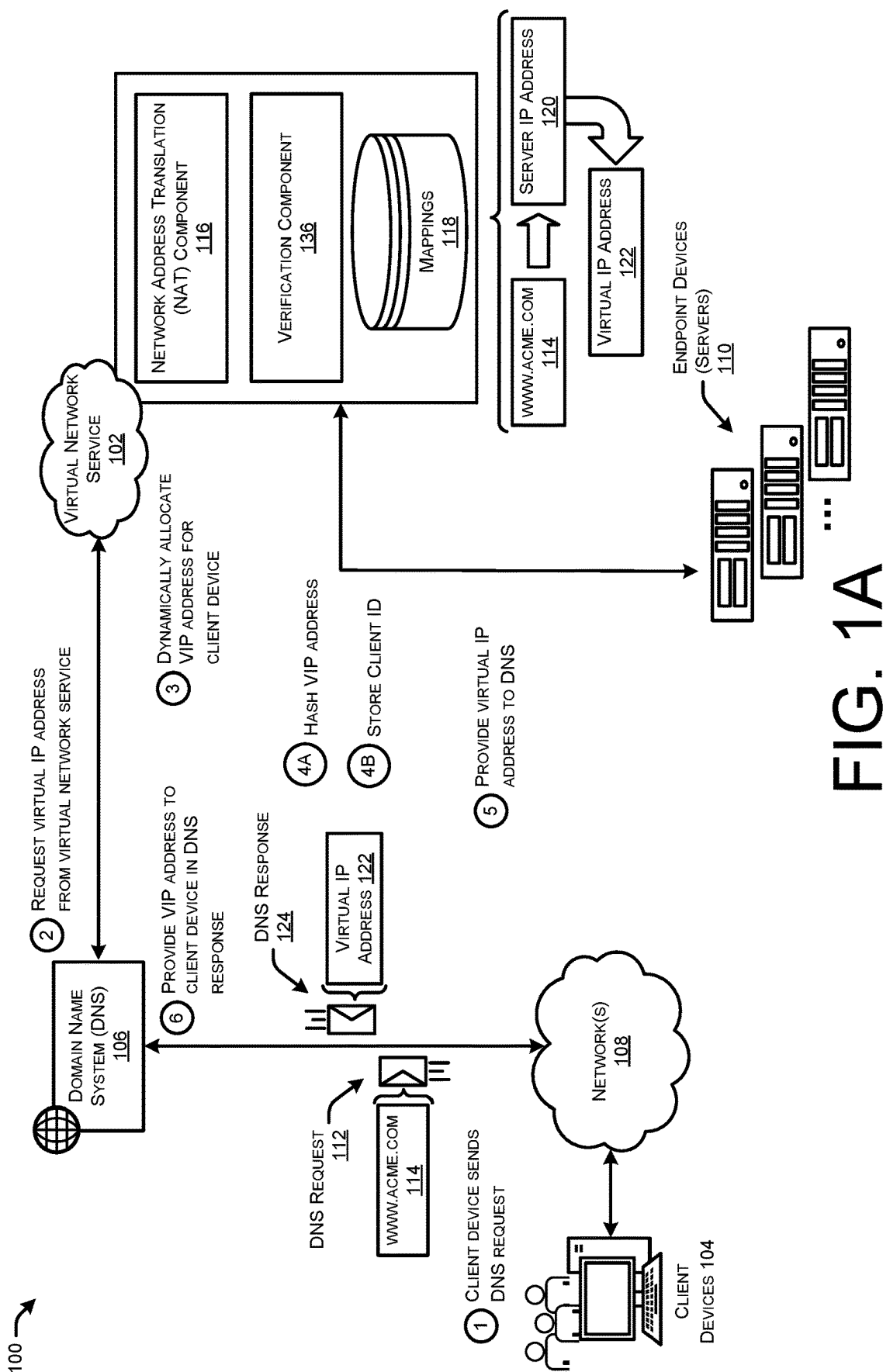
(56)

References Cited

U.S. PATENT DOCUMENTS

2019/0306188	A1	10/2019	Medvedovsky et al.
2023/0308475	A1 *	9/2023	Pane H04L 63/0435
2024/0146556	A1 *	5/2024	Joshi H04L 45/64

* cited by examiner



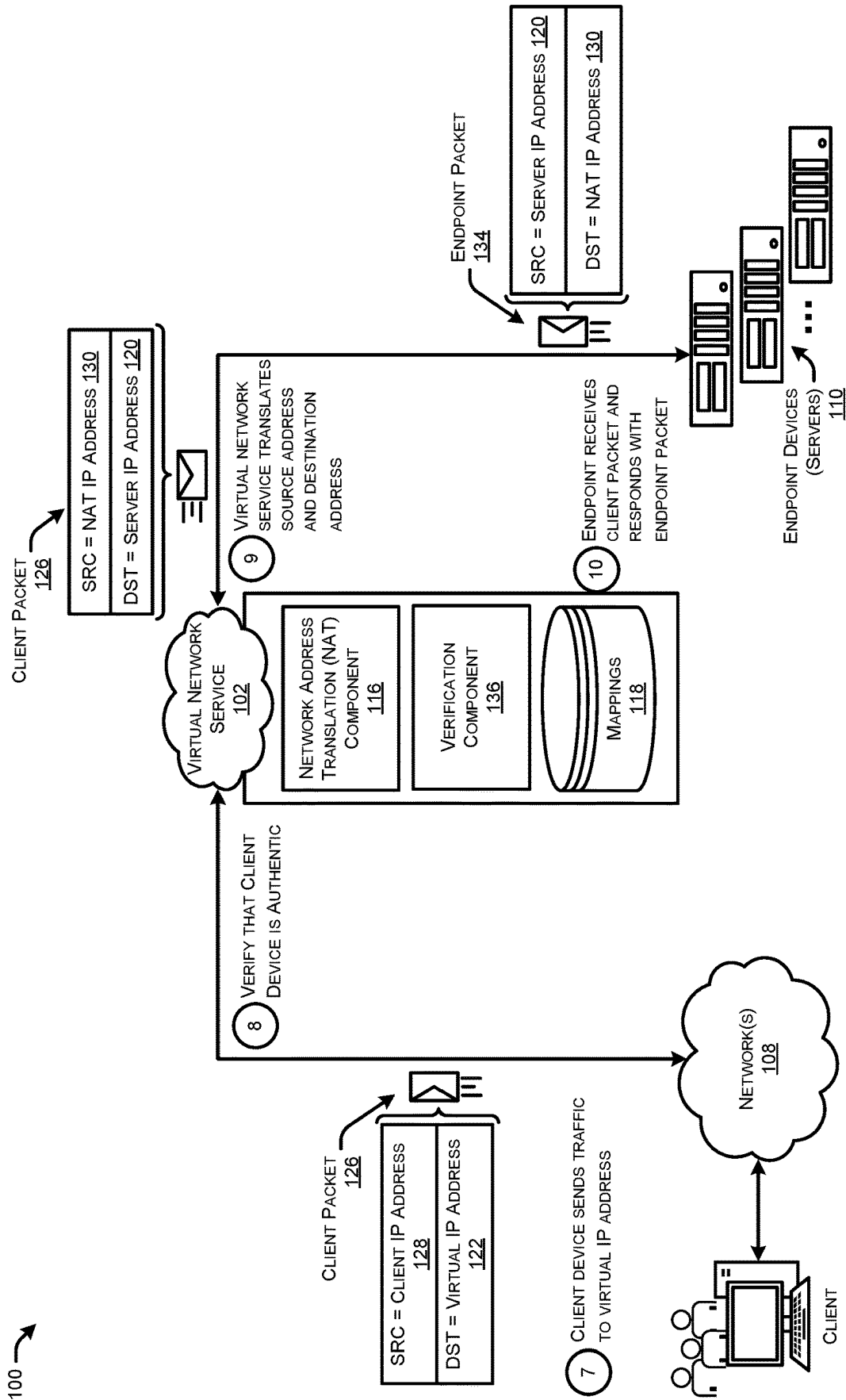


FIG. 1B

200 →

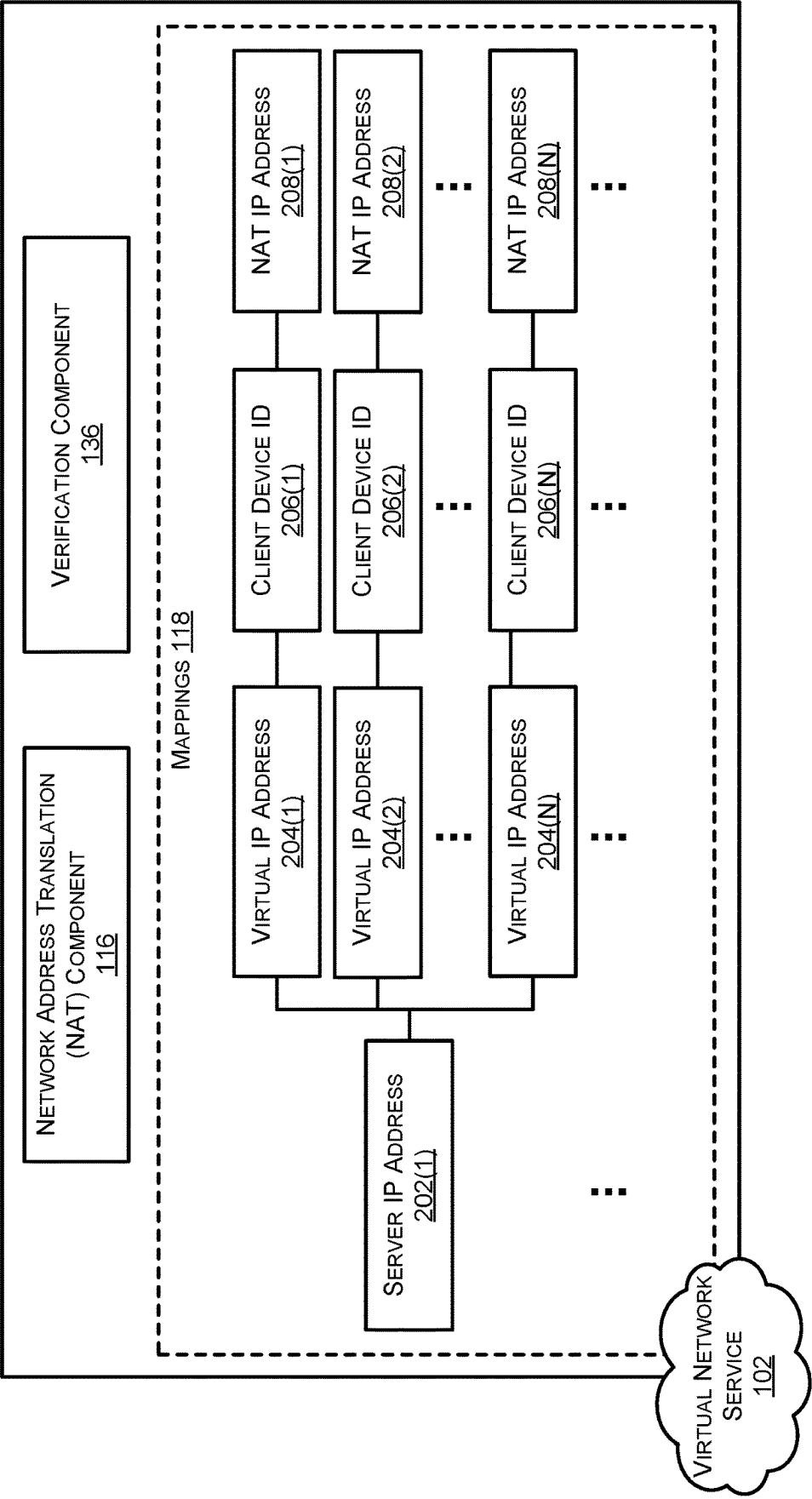


FIG. 2

300

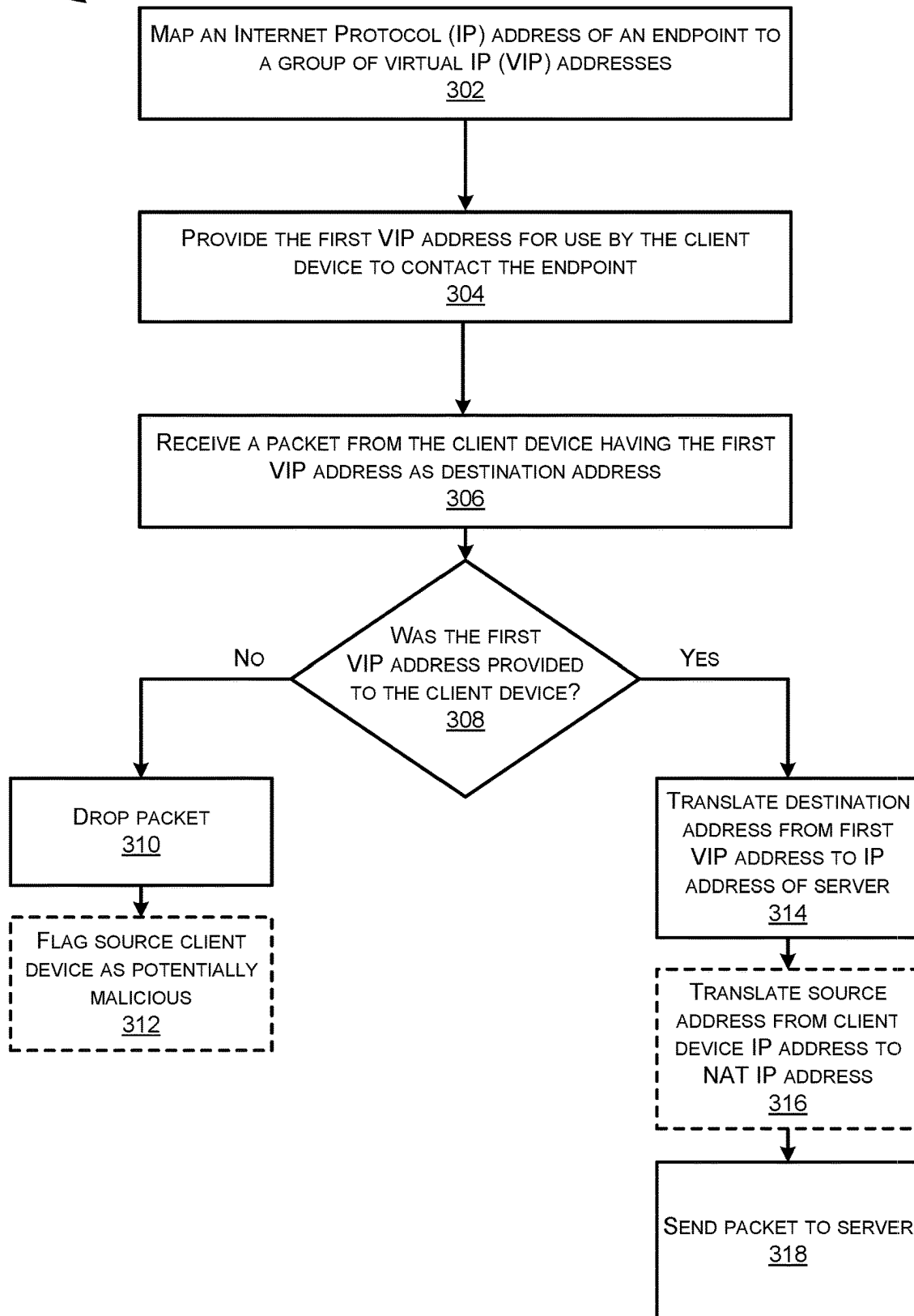


FIG. 3

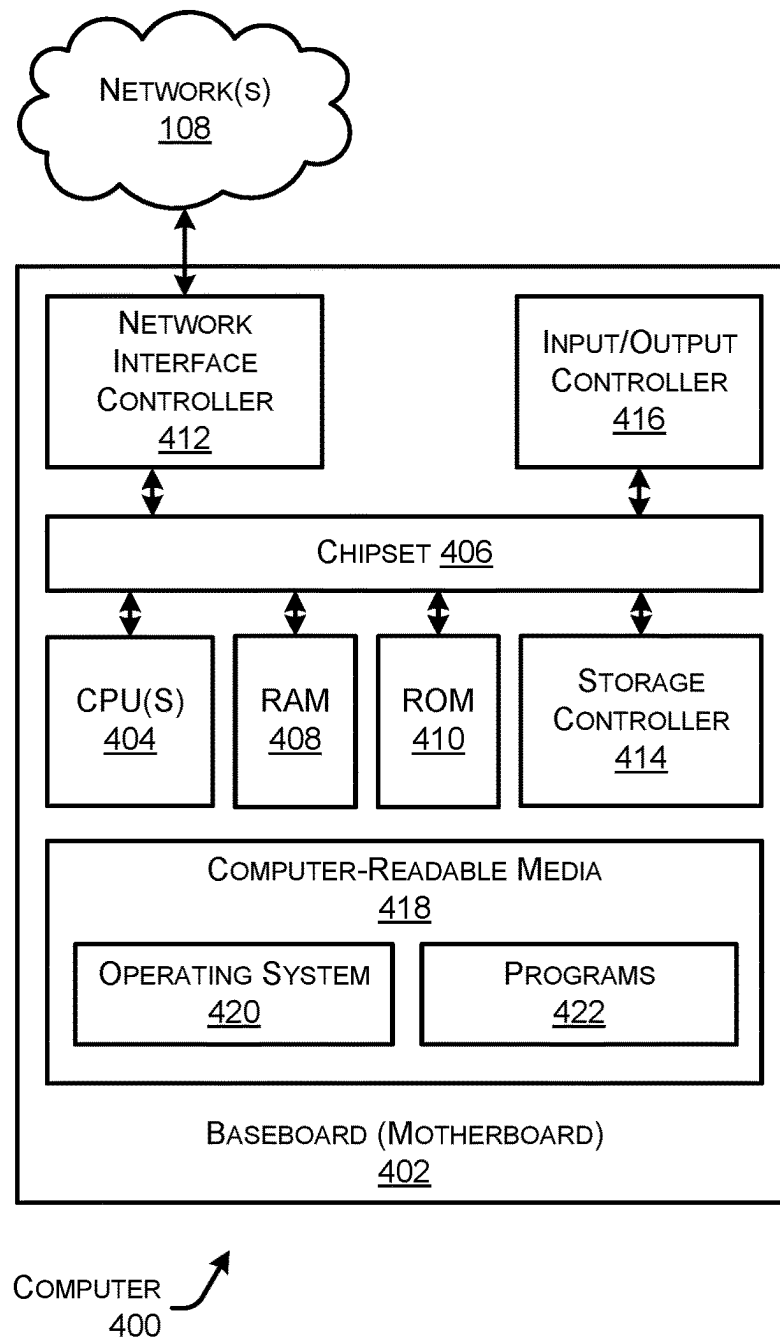


FIG. 4

1

CLIENT DEVICE VERIFICATION

TECHNICAL FIELD

The present disclosure relates generally to anonymizing addresses of endpoints, such as servers, in traffic communicated from source devices.

BACKGROUND

Networks such as the Internet use the Domain Name System (DNS) to essentially provide mappings between human-readable domain names (e.g., website addresses, service addresses, etc.) that client devices are seeking and the actual Internet Protocol (IP) addresses for devices hosting the websites or providing the services. Generally, client devices send DNS queries to resolve domain names to a DNS server, and the DNS server then resolves the domain names to the corresponding IP addresses and sends DNS responses to the client devices that include the IP addresses. The client devices are then able to communicate data packets with the desired website or service using the IP address of the device(s) supporting the website or service.

Client devices often send information in data packets that is sensitive or private, and when the data packets are sent over public networks, the information in those packets can be viewed by potentially malicious entities. Accordingly, various types of encryption protocols (e.g., Hypertext Transfer Protocol Secure (HTTPS)) are used to communicate data packets over networks such that the payload of the data packets is encrypted to prevent entities from discerning sensitive information. However, the headers of the data packets are often unencrypted because information in the headers are often needed to make networking decisions. For instance, destination IP addresses are used to make routing decisions such that the data packets reach the correct destinations, source IP addresses can be used to make firewall decisions to drop or allow packets, and so forth. However, client devices may further desire that information in the headers is obfuscated as well from potentially malicious entities, such as the source address.

In light of this, there have been large efforts to develop techniques to anonymize the client devices' IP addresses (and/or other information such as Media Access Control (MAC) addresses) to protect the privacy of users. However, the destination addresses in the packets are generally still visible because they need to be used for networking decisions, such as routing decisions. Various issues arise due to the visibility of the IP addresses shown in the destination address field. For instance, once the address of the destination device (e.g., server, endpoint, etc.) is known to a malicious entity, the address can be used for direct attacks on the destination device (e.g., distributed denial-of-service (DDOS) attacks). It can be very difficult to discriminate legitimate traffic to the servers from attacks. Additionally, malicious entities may be able to observe where the client devices are sending traffic, or which sites they are consulting. If the malicious entities are close to the client device, the malicious entities may be able to correlate the client device and server, thereby eliminating privacy of the client device and further aiding bad actors to attach a server and/or an endpoint. Thus, destination devices, such as servers or other endpoints, are vulnerable to various attacks by malicious entities.

BRIEF DESCRIPTION OF THE DRAWINGS

The detailed description is set forth below with reference to the accompanying figures. In the figures, the left-most

2

digit(s) of a reference number identifies the figure in which the reference number first appears. The use of the same reference numbers in different figures indicates similar or identical items. The systems depicted in the accompanying figures are not to scale and components within the figures may be depicted not to scale with each other.

FIG. 1A illustrates a system-architecture diagram of a virtual network service that maps virtual IP (VIP) addresses to IP addresses of endpoints devices such that a DNS server can provide client devices with respective VIP addresses that are usable to reach the endpoint devices.

FIG. 1B illustrates a system-architecture diagram of a virtual network service that verifies an authenticity of a client device and performs Network Address Translation (NAT) to convert a destination address of packets sent from a client device from a VIP address to an IP address of the endpoint device.

FIG. 2 illustrates example mappings between an IP address of a server, VIP addresses mapped to the IP address, client device identifiers (IDs) mapped to respective VIP addresses, and NAT IP addresses used for respective client devices.

FIG. 3 illustrates a flow diagram of an example method for verifying an authenticity of a client device by a virtual network service.

FIG. 4 is a computer architecture diagram showing an illustrative computer hardware architecture for implementing a computing device that can be utilized to implement aspects of the various technologies presented herein.

DESCRIPTION OF EXAMPLE EMBODIMENTS

Overview

This disclosure describes techniques for using MIPv6, NAT, LISP, and/or other techniques in conjunction with DNS to verify an authenticity of a client device attempting to send a data packet to an endpoint.

A first method to perform techniques described herein includes mapping an Internet Protocol (IP) address of an endpoint (e.g., server) to a group of virtual IP (VIP) addresses, such as allocating a block of VIP prefixes for the IP of the server. The first method may further include providing a first VIP address for use by a client device to contact the endpoint. The first method may further include receiving a packet from a source device and determining that a destination address of the packet is the first VIP address. The first method may further include determining that the source device is either 1) not the client device to which the first VIP address was provided and dropping the packet; or 2) the client device to which the first VIP address was provided and forwarding the packet to a next hop associated with the endpoint.

In some instances, the first method may further include performing Network Address Translation (NAT) by changing the destination address of the packet from the first VIP address to the IP address of the endpoint, and sending the packet to a next hop associated with the IP address of the endpoint. The method of claim 1 may be performed by a system and/or device that includes a DNS server or is associated with a DNS server.

Additionally, the techniques described herein may be performed by a system and/or device having non-transitory computer-readable media storing computer-executable

instructions that, when executed by one or more processors, performs the method described above.

EXAMPLE EMBODIMENTS

Generally, the address of a website that a client device visits is indicative of the location of the server and/or the cloud tenant that includes the server, and can be used to approximate the website that the client device is visiting. With Transport Layer Security (TLS), HTTPS, and the like, that is some of the only information available to an observer on path that may do operations like slowing or blocking some destinations, or use the information against the client in whichever fashion. While techniques have been developed to protect the source address of the client device, there are no techniques for protecting the address of the server, which is only partially hidden from public view by complex cloud architectures and load balancers. The techniques described herein obfuscate the destination IP so additional aspects of the packet become private and verify an authenticity (e.g., validate) of a source device based on the VIP used to obfuscate the destination IP.

This disclosure describes techniques for using NAT, MIP, LISP, and/or other techniques in conjunction with DNS to anonymize server-side addresses of an endpoint in data communications and verify an authenticity of a source device attempting to communicate with the endpoint. Rather than having DNS provide a client device with an IP address of an endpoint device, such as a server, the DNS instead returns a virtual IP (VIP) address that is mapped to the client device and the endpoint device. In this way, IP addresses of servers are obfuscated by a virtual network of VIP addresses. The client device may then communicate data packets to the server using the VIP address as the destination address, and a virtual network service that works in conjunction with DNS can convert the VIP address to the actual IP address of the server using NAT and forward the data packet onto the server.

In addition to obfuscating the IP addresses of servers and endpoints to prevent attacks on the servers, the techniques may further allow the virtual network service to track source devices that may be attacking the servers. For instance, the virtual network service may note the source addresses that are using VIP addresses to attempt to reach the servers despite those source devices not being provided the VIP addresses by the virtual network service.

A client device may send a DNS request to a DNS server for the DNS server to translate or resolve a domain name (e.g., website name, service name, etc.) to an IP address that is usable to contact the domain name. Traditionally, the DNS server would simply resolve the domain into the IP address (and/or other contact information) registered for the server and return the IP address to the client device. However, according to the techniques described herein, a virtual network service may work in conjunction (and/or be included in) the DNS service to provide VIP addresses on a client device-by-client device basis. That is, the virtual network service may associate a virtual network of IP addresses, or VIP addresses, for each IP address. As client devices request the DNS for the IP address of a particular server, the virtual network service and/or DNS may provide a different VIP address mapped to the IP address to each client device, and store a mapping between each VIP address and the client device that received the VIP address. Thus, client devices may request that DNS provide them with an IP address for a particular server, and each client device may

be provided with a different VIP address that is mapped to the IP address by the virtual network service.

After receiving a VIP address, the client device may then attempt to send data packets to the particular server by placing the VIP address in the destination address field of the packets. However, the VIP address may result in the data packets being steered to the virtual network service. The virtual network service may then determine that the VIP address maps to the IP address of the particular server, and may further determine that the client device was in fact provided the VIP address. For instance, in some cases the VIP address may include a hash representing the client IP address that was provided the VIP address and if the device that is attempting to use the VIP address to communicate with the endpoint has a source IP address that does not match the client IP address of the client device that was originally provided the VIP address, the virtual network service may drop the packet. In some cases, if the source IP address of the client device attempting to send the packet does match the client IP address of the client device that was provided the VIP address, then the virtual network service may forward the packet to the next hop associated with the endpoint.

In some cases, other types of information may be hashed and included in the VIP address so as to do determine that the client device that is attempting to send the packet using the VIP address was in fact provided the VIP address. For instance, the virtual network service may hash the server IP address, in which case the hash can be verified after the VIP address is translated to the server IP address (e.g., the virtual network service may perform NAT on the data packets). That is, the virtual network service may translate the VIP address in the destination field of the packets to the IP address of the particular server and send the packets to the particular server. In this way, the destination address of servers may be represented by VIP address that are not actually the IP addresses of the servers, but can be translated using NAT techniques into the correct IP addresses for reaching the servers and the virtual service can authenticate the client device sending the packet by verifying that the client device was provided the VIP address in the first place.

In some cases, the virtual network service may determine which client devices were provided which VIP addresses by storing an association between client IP addresses, client device IDs, server IP addresses, and/or VIP addresses. For instance, the virtual network service may store the association between client IP addresses, client device IDs, server IP addresses, and/or VIP addresses in a database and/or a data structure, such as a Bloom Filter. In some cases, the virtual network service may communicate with a third-party service that stores the database and/or data structure (e.g., Bloom Filter) and provide the association between client IP addresses, client device IDs, server IP addresses, and/or VIP addresses to be stored by the third-party. Once the virtual network service receives the data packet from the client device including the VIP address in the destination address, the virtual network service can compare the IP address in the source address of the data packet and determine if the source address matches any of the associations between client IP addresses, client device IDs, server IP addresses, and/or VIP addresses stored in the database and/or data structure. In cases where the database and/or the data structure are stored by a third-party, this may include the virtual network service sending a message to the third-party including the source address and an instruction to check the database and/or data structure to determine if the source address matches any of the associations between client IP addresses, client device

IDs, server IP addresses, and/or VIP addresses stored in the database and/or data structure.

In some instances, the virtual network service may additionally ensure that traffic returning to the client device and from the servers is also protected. That is, the servers may be configured to utilize their IP addresses as source addresses in data packets. In such examples, the virtual network service may also convert the source IP address of the client devices into system VIP addresses that are virtual addresses that map back to the virtual network service. Generally, the virtual network service may map a respective system VIP address to each client device such that the return traffic from the servers can be redirected back to the appropriate client device. So, when servers receive the traffic having source addresses that are system VIP addresses for the virtual network service, the servers may respond with data packets that include the respective system VIP address as a destination address. Once the virtual network service receives the packet, the virtual network service translates the destination address from the system VIP address into the IP address of the appropriate client device. In this way, the virtual network service may also obfuscate client device IP addresses while ensuring that return traffic from the servers reach the correct client device.

Certain implementations and embodiments of the disclosure will now be described more fully below with reference to the accompanying figures, in which various aspects are shown. However, the various aspects may be implemented in many different forms and should not be construed as limited to the implementations set forth herein. The disclosure encompasses variations of the embodiments, as described herein. Like numbers refer to like elements throughout.

FIG. 1A illustrates a system-architecture diagram 100 of a virtual network service 102 that maps VIP addresses to IP addresses of endpoints devices such that a DNS server can provide client devices with respective VIP addresses that are usable to reach the endpoint devices.

As illustrated, client devices 104 are able to communicate with DNS 106, which is generally one or more DNS servers that perform DNS operations. The client devices 104 may be any type of computing device that uses DNS to communicate with one or more endpoint devices 110. For instance, the client devices 104 may be personal user devices (e.g., desktop computers, laptop computers, phones, tablets, wearable devices, entertainment devices such as televisions, etc.), network devices (e.g., servers, routers, switches, access points, etc.), and/or any other type of computing devices.

The client devices 104 may communicate with the DNS 106, the virtual network service 102, the endpoint devices 110, and/or any other computing devices over one or more networks 108, such as the Internet. The network(s) 108 may each include any combination of Personal Area Networks (PANs), Local Area Networks (LANs), Campus Area Networks (CANs), Metropolitan Area Networks (MANs), extranets, intranets, the Internet, short-range wireless communication networks (e.g., ZigBee, Bluetooth, etc.) Wide Area Networks (WANs)—both centralized and/or distributed—and/or any combination, permutation, and/or aggregation thereof.

At “1,” a client device 104 may generate and send a DNS request 112 to the DNS 106. The DNS request 112 may include a request for the DNS 106 to translate a domain name (e.g., “www.acme.com”) into an IP address that can be used for sending traffic to the desired website, service, etc., associated with the domain name.

The DNS 106 may receive the DNS request 112 and, at “2,” request a VIP address from the virtual network service 102. The DNS 106 may also provide the virtual network service 102 with an indication of the domain name that the client device 104 would like translated into an IP address. In other examples, the DNS 106 may provide the actual IP address to the virtual network service 102 for the desired domain name.

Although illustrated as separate entities that communicate with each other, in some instances the DNS 106 and virtual network service 102 may be the same entity. That is, the DNS 106 may include the virtual network service 102, the virtual network service 102 may include the DNS 106, and so forth. The virtual network service 102 may be a cloud-based security service, a NAT service, and/or any type of platform or system.

The virtual network service 102 may receive the request for the VIP address for the domain name/IP address and at “3” dynamically allocate a VIP address for the client device. Generally, the virtual network service 102 may map or otherwise assign VIP addresses (e.g., virtual network) to IP addresses of endpoints 110. In some examples, the virtual network service 102 may allocate a prefix (e.g., /64 for IPV6, /24 for IPV4) in a geography to protect the addresses of the servers of endpoint device 110 that it serves in that geography. In the case of IPV4, the stat is indexed by the pair client address+virtual address, so the same virtual address can be used for multiple clients to denote different real servers of endpoint device 110 in some instances. As illustrated, the virtual network service 102 may map the domain name 114 to the service IP address 120 for the service or website, and may then map that server IP address 120 to the virtual IP address 122.

In some cases, the virtual network service may generate VIP addresses to be provided to the client devices 104 (e.g., via the DNS 106) and may check a database and/or data structure (e.g., a Bloom Filter) to determine if a generated VIP address already exists. This processes avoids collisions whereby the same VIP address is given to more than one client device. For example, the virtual network service 102 and/or a third-party may utilize a database and/or data structure and index VIP addresses and their associated server IP addresses. In some cases, the VIP address may be generated randomly and tested against the database and/or data structure for already used VIP addresses in a trial-and-error fashion, and regenerating until an unused VIP address is found.

At “4A” and/or “4B” the virtual network service 102 may hash the VIP address and/or may store client device identifying information. For instance, the virtual network service 102 may generate a hash representing a client IP address of the client device 104 and incorporate the hash into the VIP address 122. In some cases, other types of information may be hashed and included in the VIP address 122 so as to do determine that a client device that is attempting to send the packet using the VIP address 122 was in fact provided the VIP address 122. For instance, the virtual network service 102 may hash a server IP address of the endpoint 110, in which case the hash can be verified after the VIP address 122 is translated to the server IP address (e.g., the virtual network service may perform NAT on the data packets).

In some cases, providing the VIP address 122 to the client device 104 may cause the virtual network service 102 to store client ID data (e.g., an association between client device IP addresses, client device IDs, server IP addresses, and/or VIP addresses) in a database and/or a data structure, such as a Bloom Filter, based on the received the DNS

7

request **112**. In this way, the VIP address **122** acts as a cookie enabling the virtual network service **102** to recall the client ID data when the VIP address **122** is received back from the client device as a destination address. In some cases, the virtual network service **102** may communicate with a third-party service that stores the database and/or data structure (e.g., Bloom Filter) and provide the association between client device IP addresses, client device IDs, server IP addresses, and/or VIP addresses to be stored by the third-party.

At “5,” the virtual network service **102** may provide the VIP address to the DNS and/or directly to the client device **104**. At “6,” the VIP address **122** may be provided to the client device **104** in a DNS response **124** such that the client device **104** determines that the VIP address is usable to contact the desired endpoint device **110**. Although illustrated as servers, the endpoint devices **110** can generally be any device that a client device **104** would like to contact, including user devices (e.g., laptops, phones, wearable devices, etc.), network devices (e.g., routers, switches, hubs, etc.), and/or any other type of devices that can be contacted over networks **108**.

FIG. 1B illustrates a system-architecture diagram **100** of a virtual network service **102** that performs verification of an authenticity of a client device **104** and performs Network Address Translation (NAT) to convert a destination address of packets sent from a client device **104** from a VIP address to an IP address of the endpoint device **110**.

At “7,” the client device **104** may send traffic including a client packet **126** to the VIP address **122** that was provided in the DNS response **124**. The client packet **126** may include the VIP address **122** corresponding to the desired endpoint device **110** as the destination address, and may further include the client IP address **128** of the client device **104** as the source address. Generally, the VIP address **122** may cause the client packet **126** to be communicated to the virtual network service **102** by devices in the network(s) **108**.

At “8,” the virtual network service **102** may verify that the client device **104** is authentic. For example, a verification component **136** of the virtual network service **102** may determine that the VIP address **122** maps to the IP address of the particular server (e.g., server IP address **120**), and may further determine that the client device **104** was in fact provided the VIP address **122**. For instance, in some cases the VIP address **122** may include a hash representing the client IP address **128** that was provided the VIP address **122** and if the source device (e.g., client device **104**) that is attempting to use the VIP address **122** to communicate with the endpoint **110** has a source IP address that does not match the client IP address **128** of the client device **104** that was originally provided the VIP address **122**, the virtual network service **102** may drop the client packet **126**. In some cases, if the source IP address of the client device attempting to send the client packet **126** does match the client IP address **128** of the client device **104** that was provided the VIP address **122**, then the virtual network service **102** may forward the packet to the next hop associated with the endpoint **110**.

In some cases, other types of information may be hashed and included in the VIP address **122** so as to do determine that the client device **104** that is attempting to send the client packet **126** using the VIP address **122** was in fact provided the VIP address **122**. For instance, the virtual network service **102** may hash the server IP address **120**, in which case the hash can be verified after the VIP address **122** is

8

translated to the server IP address **120** (e.g., the virtual network service **102** may perform NAT on the data client packet **126**).

In some cases, the verification component **136** of the virtual network service **102** may determine which client devices were provided which VIP addresses by storing an association between client IP addresses, client device IDs, server IP addresses, and/or VIP addresses. For instance, the virtual network service **102** may store the association between client IP addresses, client device IDs, server IP addresses, and/or VIP addresses in a database and/or a data structure, such as a Bloom Filter. In some cases, the virtual network service **102** may communicate with a third-party service that stores the database and/or data structure (e.g., Bloom Filter) and provide the association between client IP addresses, client device IDs, server IP addresses, and/or VIP addresses to be stored by the third-party. Once the virtual network service **102** receives the client packet **126** from the client device **104** including the VIP address **122** in the destination address, the verification component **136** of the virtual network service **102** can compare the client IP address **128** in the source address of the client packet **126** and determine if the source address matches any of the associations between client IP addresses, client device IDs, server IP addresses, and/or VIP addresses stored in the database and/or data structure. In cases where the database and/or the data structure are stored by a third-party, this may include the virtual network service **102** sending a message to the third-party including the source address and an instruction to check the database and/or data structure to determine if the source address matches any of the associations between client IP addresses, client device IDs, server IP addresses, and/or VIP addresses stored in the database and/or data structure. In response to a match being found, the virtual network service **102** may forward the packet to the next hop associated with the endpoint **110**. In response to a match not being found, the virtual network service **102** may drop the packet.

At “9,” the virtual network service **102** may translate the source address and the destination address of the client packet **126** using NAT techniques. For instance, a NAT component **116** may utilize mappings **118** to determine how to translate or convert the addresses. Specifically, the NAT component **116** may translate the VIP address **122** to the actual service IP address **120** of the server/endpoint device **110** and place the server IP address **120** in the destination address of the client packet **126**.

In some examples, optionally, the virtual network service **102** may additionally performing NAT techniques to translate the source address of the client packet **126** such that return traffic from the endpoint devices **110** is sent to the virtual network service **102**. For instance, the NAT component **116** may utilize VIP addresses (NAT IP addresses) corresponding to the virtual network service **102** as source addresses such that return traffic is sent to the virtual network service **102**. The NAT component **116** may translate the client IP address **128** in the source address field of the packet **126** into a particular NAT IP address **130**, and store a mapping **118** between the client device **104** and the NAT IP address **130**. In some instances, client devices **104** may have respective NAT IP addresses **130** such that the virtual network service **102** can cause return traffic from the endpoint devices **110** flow back through the virtual network service **102**. Thus, the NAT component **116** may translate the source address of the client packet **126** to a NAT IP address **130** that corresponds to the virtual network service **102**, and

is mapped in the mappings **118** to a client ID of the client device **104** (e.g., the client IP address **128** in some instances).

The client packet **126** is then communicated to the endpoint device **110** corresponding to the server IP address **120** (and the corresponding VIP address **122**) indicated in the client packet **126**. The endpoint device **110** may, in some examples, determine to respond to the client device **104**. The endpoint device **110** may use the source address of the client packet **126** (e.g., the NAT IP address **130**) as the destination address in the endpoint packet **134**, and the server IP address **132** as the source address. Thus, the endpoint device **110** may send one or more endpoint packets **134** with the NAT IP address **130** as the destination address such that the endpoint packet **134** is sent to the virtual network service **102** at “10.”

The NAT component **116** may translate the source IP address and the destination IP address in the endpoint packet **134**. The NAT component **116** may use the mappings **118** to translate the NAT IP address **130** in the destination address field to the client IP address **128** of the client device **104** such that the endpoint packet **134** is sent to the client device **104**. Further, the NAT component **116** may use the mappings **118** to translate the server IP address **120** in the source address field to the virtual IP address **122** that is allocated by the virtual network service **102**. In this way, any return traffic from the client device **104** may be sent to the VIP address **122** that is the source address of the endpoint packet **134**. The endpoint packet **134** may then be sent to the client device **104**.

In some cases, the virtual network service **102** may use a Home Address option of Mobile IP version 6 (MIPv6) to allow a server to communicate with a home agent of a client device. For example, the techniques may be performed according to home agent techniques of Mobile IPv6 defined by Request for Comments (RFC) **6275**. The DNS **106** may use a new interface with a HA function where the interface enables the DNS server **106** to query the HA function for a virtual address upon a DNS request. The novel query contains the client address and the server address that it would have returned by the DNS server without the invention, for its customers, the HA function returns an address from the /64 in the virtual interface. The end result is that the set of clients of the virtual network service **102** appear to be located in the same flat subnet, with the effect to hide the topology and the server addresses altogether.

Using that interface, when a new DNS request is made (either by new client **104** or for a different server from a known client **104**), one virtual address (serving as home address in mobility management) in the /64 is allocated dynamically by the HA, returned to the DNS server **106** and that may be the address returned to the client in the DNS response. When the same client does the same request again, it may be preferable to return the same address as the first time to keep the sessions going. To that effect, the techniques described herein may include load-balancing the DNS **106** to HA interface based on the client address, or a hash of the client address and the DNS name. This way the same request may be served by the same HA server which can return the virtual IP address from an existing state if one is found.

A home agent function (from MIPv6 RFC **6275** or similar mobility protocol) may be associated to the virtual network service **102**. That home agent function may tunnel the packet for the virtual address assigned to the server to the real address of the server (used as Care-Of Address in the packet). MIPv6 may use a normal tunnel but alternates like SRH insertion such that NAT and/or Prefix Address Trans-

lation (PAT) are possible. In the case of NAT, the home agent may also NAT the source address to another address it owns, so as to be on path of the traffic back.

In the case of MIPv6, the server of endpoint device **110** may use Home Address options to communicate to the home agent with respect to the particular client device **104**. In-band (e.g., in the tunnel as an implicit function) or out of band, the home agent may instruct the server of endpoint device **110** to support the home address, e.g., add it to a loop back or store it in the socket information. When it responds, the server of endpoint device **110** may respond with that virtual HA IP address. If ingress filtering—BCP **38**. RFC **2827**—is enforced, the server may tunnel back to the home agent, using its real address as source. This enables the server of endpoint device **110** to perform as normal but based on the virtual IP, so any application state and crypto that relies on that address continues to work. In some cases the stack may absorb the addresses and only present the real server and client addresses to the upper layers, in a fashion similar—though stateless in this case—of host identity protocol (HIP). In the case of IPV4, a NAT function in the cloud service may provide the same result, which can be achieved without tunneling.

The home agent may be located in the virtual network service **102**. Packets to and from the home agent and the server may be tunneled using the Home Address option from the server to the home agent, in which case the source may be the server and the destination may be the home agent with no encapsulation. To hide the IP address of the server (e.g., the Care-of Address in MIPv6 terms), the server of endpoint device **110** may answer to the client device **104** using the home address (e.g., virtual IP address **122**) as a source address and the client IP address **128** as a destination a destination address. In this example, there is no destination option and no encapsulation.

FIG. **2** illustrates example mappings **118** between an IP address of a server, VIP addresses mapped to the IP address, client device identifiers (IDs) mapped to respective VIP addresses, and NAT IP addresses used for respective client devices. As illustrated, the NAT component **116** may store mappings, which may generally be any type of association in memory. Although not illustrated, at least some of the mapping **118** information may be stored at the DNS **106**, and/or the DNS **106** may be included in the virtual network service **102**. The mappings **118** may include a database and/or data structure (e.g., such as a Bloom Filter) usable by the virtual network service to verify the authenticity of client devices, such as the client device **104**, using the techniques discussed herein.

The NAT component **116** may initially store indications of service IP addresses **202(1)-202(N)** (not illustrated). Generally, each endpoint device **110**, or server, has at least one actual server IP address **202**. Each server IP address **202** may then be mapped to, or allocated, a virtual network of VIP addresses **204(1)-204(N)** where “N” is based on the number of client devices **104** that have requested the IP address for the particular server of endpoint device **110**.

Each time the NAT component **116** allocates and provides a client device **104** with a VIP address **204**, the NAT component **116** may store an association or mapping **118** between the VIP address **204** and a client device ID **206** for that client device **104** (E.g., client IP address in some examples). Thus, each VIP address **204** is associated with a respective client device ID **206**. In some instances, the NAT component **116** may further map each client device ID **206** with a respective NAT IP address **208** that is used for ensuring that return traffic from the servers of endpoint

11

device 110 is sent back to the virtual network service 102. By having a NAT IP address 208 mapped to each client device ID 206, when the virtual network service 102 receives traffic back from the endpoint devices 110 that has the destination address as a particular NAT IP address 208, the NAT component 116 may use the mappings 118 to translate the NAT IP address 208 back to a client IP address of the client device 104 and send the packet to the correct client device 104 using the current client IP address.

It should be understood that the mappings 118 are merely illustrative, and other types of mappings 118 or data structures can be used for the NAT techniques described herein.

FIG. 3 illustrates a flow diagram of an example method 300 that illustrates aspects of the functions performed at least partly by the devices as described in FIGS. 1A, 1B, and 2. The logical operations described herein with respect to FIG. 3 may be implemented (1) as a sequence of computer-implemented acts or program modules running on a computing system and/or (2) as interconnected machine logic circuits or circuit modules within the computing system.

The implementation of the various components described herein is a matter of choice dependent on the performance and other requirements of the computing system. Accordingly, the logical operations described herein are referred to variously as operations, structural devices, acts, or modules. These operations, structural devices, acts, and modules can be implemented in software, in firmware, in special purpose digital logic, and any combination thereof. It should also be appreciated that more or fewer operations might be performed than shown in the FIG. 3 and described herein. These operations can also be performed in parallel, or in a different order than those described herein. Some or all of these operations can also be performed by components other than those specifically identified. Although the techniques described in this disclosure is with reference to specific components, in other examples, the techniques may be implemented by less components, more

FIG. 3 illustrates a flow diagram of an example method 300 for using Network Address Translation (NAT) in conjunction with DNS to verify an authenticity (e.g., validate) of a client device during data communications.

In some examples, the steps of method 300 may be performed, at least partly, by a virtual network service 102, which may include, be included in, or at least be associated with a DNS 106 (e.g., communicatively coupled). The virtual network service 102 may comprise one or more processors and one or more non-transitory computer-readable media storing computer-executable instructions that, when executed by the one or more processors, cause the one or more processors to perform the operations of method 300.

At 302, the virtual network service 102 may map an IP address of an endpoint to a group of VIP addresses. For instance, the NAT component 116 may allocate a virtual network, which may be the group of VIP addresses, for an IP address of a server of endpoint device 110.

At 304, the virtual network service 102 may provide a first VIP address for use by a client device to contact the endpoint. For instance, the virtual network service 102 may provide the VIP address 122 to the DNS 106, which in turn provides the VIP address 122 to the client device 104 in a DNS response 124.

At 306, the virtual network service 102 may receive a packet from the client device that has the first VIP address as the destination address. For instance, the virtual network service 102 may receive a client packet 126 from the client device 104 that includes the VIP address 122 that maps to

12

the IP address 120 of the endpoint/server of endpoint device 110 that the client device 104 would like to communicate with.

At 308, the virtual network service 102 may determine whether the first VIP address was provided to the client device 314. For instance, the NAT component 116 may check the mappings 118 to see if the VIP address 122 was provided to the client device 104 and is mapped to the client device ID.

In instances where the first VIP address was not provided to the client device (e.g., rogue device), the virtual network service 102 may, at 316 drop the packet because the device is not permitted to use that VIP address. Further, the virtual network service 102 may also flag the source device as potentially malicious at 318 because the source device was using a VIP address that was not provided to it by the virtual network service 102, and it may have been improperly obtained by a malicious observer.

In instances where the first VIP address was provided to the client device 104, the virtual network service 102 may translate the destination address from the first VIP address to an IP address of the server at 310. Further, the virtual network service 102 may, optionally, translate the source address from being the client device IP address 128 to the NAT IP address 130 for return traffic from the server/endpoint 110. At 324, the virtual network service 102 may then send the client packet 126 to the endpoint device 110.

FIG. 4 shows an example computer architecture for a computer 400 capable of executing program components for implementing the functionality described above. The computer architecture shown in FIG. 4 illustrates a conventional server computer, workstation, desktop computer, laptop, tablet, network appliance, e-reader, smartphone, or other computing device, and can be utilized to execute any of the software components presented herein. The computer 400 may, in some examples, correspond to a device and/or a system of devices that support the virtual network service 102, the DNS 106, and/or a combination thereof, described herein, and may comprise networked devices such as servers, switches, routers, hubs, bridges, gateways, modems, repeaters, access points, etc. That is, the computer 400, or a system of computers 400, may be configured to perform some or all of the techniques performed by the virtual network service 102, some or all of the techniques performed by the DNS 106, and/or any combination thereof.

The computer 400 includes a baseboard 402, or “motherboard,” which is a printed circuit board to which a multitude of components or devices can be connected by way of a system bus or other electrical communication paths. In one illustrative configuration, one or more central processing units (“CPUs”) 404 operate in conjunction with a chipset 406. The CPUs 404 can be standard programmable processors that perform arithmetic and logical operations necessary for the operation of the computer 400.

The CPUs 404 perform operations by transitioning from one discrete, physical state to the next through the manipulation of switching elements that differentiate between and change these states. Switching elements generally include electronic circuits that maintain one of two binary states, such as flip-flops, and electronic circuits that provide an output state based on the logical combination of the states of one or more other switching elements, such as logic gates. These basic switching elements can be combined to create more complex logic circuits, including registers, adders-subtractors, arithmetic logic units, floating-point units, and the like.

13

The chipset **406** provides an interface between the CPUs **404** and the remainder of the components and devices on the baseboard **402**. The chipset **406** can provide an interface to a RAM **408**, used as the main memory in the computer **400**. The chipset **406** can further provide an interface to a computer-readable storage medium such as a read-only memory (“ROM”) **410** or non-volatile RAM (“NVRAM”) for storing basic routines that help to startup the computer **400** and to transfer information between the various components and devices. The ROM **410** or NVRAM can also store other software components necessary for the operation of the computer **400** in accordance with the configurations described herein.

The computer **400** can operate in a networked environment using logical connections to remote computing devices and computer systems through a network, such as the network **108**. The chipset **406** can include functionality for providing network connectivity through a NIC **412**, such as a gigabit Ethernet adapter. The NIC **412** is capable of connecting the computer **400** to other computing devices over the network **108**. It should be appreciated that multiple NICs **412** can be present in the computer **400**, connecting the computer to other types of networks and remote computer systems.

The computer **400** can be connected to a storage device **418** that provides non-volatile storage for the computer. The storage device **418** can store an operating system **420**, programs **422**, and data, which have been described in greater detail herein. The storage device **418** can be connected to the computer **400** through a storage controller **414** connected to the chipset **406**. The storage device **418** can consist of one or more physical storage units. The storage controller **414** can interface with the physical storage units through a serial attached SCSI (“SAS”) interface, a serial advanced technology attachment (“SATA”) interface, a fiber channel (“FC”) interface, or other type of interface for physically connecting and transferring data between computers and physical storage units.

The computer **400** can store data on the storage device **418** by transforming the physical state of the physical storage units to reflect the information being stored. The specific transformation of physical state can depend on various factors, in different embodiments of this description. Examples of such factors can include, but are not limited to, the technology used to implement the physical storage units, whether the storage device **418** is characterized as primary or secondary storage, and the like.

For example, the computer **400** can store information to the storage device **418** by issuing instructions through the storage controller **414** to alter the magnetic characteristics of a particular location within a magnetic disk drive unit, the reflective or refractive characteristics of a particular location in an optical storage unit, or the electrical characteristics of a particular capacitor, transistor, or other discrete component in a solid-state storage unit. Other transformations of physical media are possible without departing from the scope and spirit of the present description, with the foregoing examples provided only to facilitate this description. The computer **400** can further read information from the storage device **418** by detecting the physical states or characteristics of one or more particular locations within the physical storage units.

In addition to the mass storage device **418** described above, the computer **400** can have access to other computer-readable storage media to store and retrieve information, such as program modules, data structures, or other data. It should be appreciated by those skilled in the art that com-

14

puter-readable storage media is any available media that provides for the non-transitory storage of data and that can be accessed by the computer **400**. In some examples, the operations performed by devices in the virtual network service **102**, and or any components included therein, may be supported by one or more devices similar to computer **400**. Stated otherwise, some or all of the operations performed by the virtual network service **102**, and or any components included therein, may be performed by one or more computer devices **400** operating in a cloud-based arrangement.

By way of example, and not limitation, computer-readable storage media can include volatile and non-volatile, removable and non-removable media implemented in any method or technology. Computer-readable storage media includes, but is not limited to, RAM, ROM, erasable programmable ROM (“EPROM”), electrically-erasable programmable ROM (“EEPROM”), flash memory or other solid-state memory technology, compact disc ROM (“CD-ROM”), digital versatile disk (“DVD”), high definition DVD (“HD-DVD”), BLU-RAY, or other optical storage, magnetic cassettes, magnetic tape, magnetic disk storage or other magnetic storage devices, or any other medium that can be used to store the desired information in a non-transitory fashion.

As mentioned briefly above, the storage device **418** can store an operating system **420** utilized to control the operation of the computer **400**. According to one embodiment, the operating system comprises the LINUX operating system. According to another embodiment, the operating system comprises the WINDOWS® SERVER operating system from MICROSOFT Corporation of Redmond, Washington. According to further embodiments, the operating system can comprise the UNIX operating system or one of its variants. It should be appreciated that other operating systems can also be utilized. The storage device **418** can store other system or application programs and data utilized by the computer **400**.

In one embodiment, the storage device **418** or other computer-readable storage media is encoded with computer-executable instructions which, when loaded into the computer **400**, transform the computer from a general-purpose computing system into a special-purpose computer capable of implementing the embodiments described herein. These computer-executable instructions transform the computer **400** by specifying how the CPUs **404** transition between states, as described above. According to one embodiment, the computer **400** has access to computer-readable storage media storing computer-executable instructions which, when executed by the computer **400**, perform the various processes described above with regard to FIGS. 1-3B. The computer **400** can also include computer-readable storage media having instructions stored thereupon for performing any of the other computer-implemented operations described herein.

The computer **400** can also include one or more input/output controllers **416** for receiving and processing input from a number of input devices, such as a keyboard, a mouse, a touchpad, a touch screen, an electronic stylus, or other type of input device. Similarly, an input/output controller **416** can provide output to a display, such as a computer monitor, a flat-panel display, a digital projector, a printer, or other type of output device. It will be appreciated that the computer **400** might not include all of the components shown in FIG. 4, can include other components that are not explicitly shown in FIG. 4, or might utilize an architecture completely different than that shown in FIG. 4.

15

The computer **400** may include one or more hardware processors **404** (processors) configured to execute one or more stored instructions. The processor(s) **404** may comprise one or more cores. Further, the computer **400** may include one or more network interfaces configured to provide communications between the computer **400** and other devices, such as the communications described herein as being performed by the virtual network service **102**. The network interfaces may include devices configured to couple to personal area networks (PANs), wired and wireless local area networks (LANs), wired and wireless wide area networks (WANs), and so forth. For example, the network interfaces may include devices compatible with Ethernet, Wi-Fi™, and so forth.

The programs **422** may comprise any type of programs or processes to perform the techniques described in this disclosure for providing a virtual network service **102** that is capable of anonymizing service-side addresses. That is, the computer **400** may comprise any one of the devices that support the virtual network service **102**. The programs **422** may comprise any type of program that cause the computer **400** to perform techniques for communicating with other devices using any type of protocol or standard, and performing NAT and/or DNS, to anonymize server-side addresses.

While the invention is described with respect to the specific examples, it is to be understood that the scope of the invention is not limited to these specific examples. Since other modifications and changes varied to fit particular operating requirements and environments will be apparent to those skilled in the art, the invention is not considered limited to the example chosen for purposes of disclosure, and covers all changes and modifications which do not constitute departures from the true spirit and scope of this invention.

Although the application describes embodiments having specific structural features and/or methodological acts, it is to be understood that the claims are not necessarily limited to the specific features or acts described. Rather, the specific features and acts are merely illustrative some embodiments that fall within the scope of the claims of the application.

What is claimed is:

1. A system associated with a Domain Name System (DNS) service that anonymizes Internet Protocol (IP) addresses of endpoints using groups of virtual IP (VIP) addresses, the system comprising:

- one or more processors; and
- one or more non-transitory computer-readable media storing computer-executable instructions that, when executed by the one or more processors, cause the one or more processors to perform operations comprising: mapping an IP address of an endpoint to a group of VIP addresses;
- generating a hash representing at least one of a client IP address associated with a client device and the IP address of the endpoint;
- incorporating at least a portion of the hash into a first VIP address;
- providing the first VIP address for use by the client device to contact the endpoint;
- receiving a packet from a source device;
- determining that a destination address of the packet is the first VIP address; and
- determining that the source device is:
 - not the client device to which the first VIP address was provided and dropping the packet based at least in

16

part on invalidating the client device based at least in part on the first VIP address not including the hash; or

the client device to which the first VIP address was provided and forwarding the packet to a next hop associated with the endpoint based at least in part on validating the client device based at least in part on the first VIP address including the hash.

2. The system of claim 1, the operations further comprising:

- determining that the first VIP address includes the hash; and
- validating the client device based at least in part on the first VIP address including the hash.

3. The system of claim 1, the operations further comprising:

- generating the first VIP address;
- accessing a Bloom Filter to determine if the first VIP address already exists;
- determining that the first VIP address is not stored in the Bloom Filter; and
- providing the first VIP address for use by the client device to contact the endpoint in response to determining that the first VIP address does not already exist.

4. The system of claim 1, the operations further comprising:

- generating the first VIP address;
- accessing a Bloom Filter to determine if the first VIP address already exists;
- determining that the first VIP address is stored in the Bloom Filter; and
- generating a second VIP address in response to determining that the first VIP address already exists.

5. The system of claim 1, the operations further comprising:

- generating a hash representing a client IP address associated with the client device and at least one of: the IP address of the endpoint;
- a Home Address associated with the system; or
- a Care-of address associated with the endpoint; and
- incorporating at least a portion of the hash into the first VIP address.

6. The system of claim 1, further comprising:

- generating a hash associated with a client IP address associated with the client device;
- storing hash data associated with the hash in a Bloom Filter; and

in response to receiving the packet from the source device, accessing the Bloom Filter to determine if a client ID of the source device matches the hash data.

7. The system of claim 6, wherein the Bloom Filter is stored by a third-party service.

8. The system of claim 1, wherein the IP address of the client device comprises a first portion including a prefix and a second portion including bits.

9. A method performed at least partly by a computing system associated with a Domain Name System (DNS) service that anonymizes Internet Protocol (IP) addresses of endpoints using groups of virtual IP (VIP) addresses, the method comprising:

- mapping an IP address of an endpoint to a group of VIP addresses;
- generating a hash representing at least one of a client IP address associated with a client device;
- incorporating at least a portion of the hash into a first VIP address;

17

providing the first VIP address for use by the client device to contact the endpoint;
 storing client ID data associated with the client device in response to providing the first VIP address to the client device;
 receiving a packet from a source device;
 determining that a destination address of the packet is the first VIP address;
 determining that the source device is the client device to which the first VIP address was provided based at least in part on validating the client device based at least in part on the first VIP address including the hash; and
 forwarding the packet to a next hop associated with the endpoint.

10. The method of claim 9, wherein the first VIP address comprises a cookie and receiving the packet from the source device comprises accessing the cookie.

11. The method of claim 9, further comprising:
 storing the client ID data in a Bloom Filter; and
 in response to receiving the packet from the source device, accessing the Bloom Filter to determine if a client ID of the source device matches the client ID data.

12. The method of claim 11, further comprising, prior to sending the packet:

generating the hash representing at least the client IP address associated with the client device and the IP address of the endpoint; and
 incorporating at least a portion of the hash into the first VIP address.

13. The method of claim 12, further comprising:
 determining that the first VIP address includes the hash; and
 validating the client device based at least in part on the first VIP address including the hash.

14. The method of claim 9, further comprising:
 generating the first VIP address;
 accessing a Bloom Filter to determine if the first VIP address already exists;
 determining that the first VIP address is stored in the Bloom Filter; and
 generating a second VIP address in response to determining that the first VIP address already exists.

15. The method of claim 9, further comprising:
 generating a hash representing a client IP address associated with the client device and at least one of:
 the IP address of the endpoint;
 a Home Address associated with the computing system; or
 a Care-of address associated with the endpoint; and
 incorporating at least a portion of the hash into the first VIP address.

18

16. The method of claim 9, wherein the computing system includes a Network Address Translation (NAT) service and the DNS service.

17. One or more non-transitory computer-readable media storing computer executable instructions that, when executed by one or more processors, cause the one or more processors to perform operations comprising:

mapping an IP address of an endpoint to a group of VIP addresses;

generating a hash representing at least one of a client IP address associated with a client device;

incorporating at least a portion of the hash into a first VIP address;

providing the first VIP address for use by the client device to contact the endpoint;

storing client ID data associated with the client device in response to providing the first VIP address to the client device;

receiving a packet from a source device;

determining that a destination address of the packet is the first VIP address;

determining that the source device is not the client device to which the first VIP address was provided based at least in part on invalidating the client device based at least in part on the first VIP address not including the hash; and
 dropping the packet.

18. The one or more non-transitory computer-readable media of claim 17, further comprising:

generating the hash representing a client IP address associated with the client device and at least one of:

the IP address of the endpoint;

a Home Address associated with a computing system; or

a Care-of address associated with the endpoint; and

incorporating at least a portion of the hash into the first VIP address.

19. The one or more non-transitory computer-readable media of claim 17, further comprising:

storing the client ID data in a Bloom Filter; and

in response to receiving the packet from the source device, accessing the Bloom Filter to determine if a client ID of the source device matches the client ID data.

20. The one or more non-transitory computer-readable media of claim 19, further comprising, prior to sending the packet:

generating a hash representing at least one of a client IP address associated with the client device or the IP address of the endpoint; and

incorporating at least a portion of the hash into the first VIP address.

* * * * *